PROMPT:
You are a senior computer-vision scientist, TensorFlow engineer, peer reviewer, and technical educator with expertise in CRISP-DM and reproducible machine learning.

I have attached an executed image-classification notebook and its exported results. The project compares a custom CNN with a fine-tuned MobileNetV2 on the Intel Image Classification dataset.

Your task is to conduct a rigorous, evidence-based audit and improvement of this project. Do not fabricate or estimate any results. Treat the executed notebook, CSV files, JSON summary, and images as the only authoritative evidence. If two artifacts disagree, identify the inconsistency explicitly and determine which result should be used.

Work through these CRISP-DM stages:

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Deployment

For every stage:

- Identify what the notebook did correctly.
- Identify technical weaknesses or missing considerations.
- Explain risks such as leakage, overfitting, nondeterminism, class ambiguity, calibration, and distribution shift.
- Recommend specific improvements.
- Cite the exact notebook cell, metric, table, or image supporting each conclusion.
- Clearly distinguish measured evidence, inference, and recommendation.

Evaluate the custom CNN and MobileNetV2 using test accuracy, loss, class-level precision, recall, F1, macro averages, confusion patterns, confident mistakes, and Grad-CAM. Explain why transfer learning performed better without claiming that model complexity alone caused the improvement.

End with:

1. A verified results table
2. Five principal findings
3. Five limitations
4. Five technically feasible improvements
5. A deployment and monitoring plan

6. A reproducibility checklist
7. Questions that cannot be answered from the existing artifacts

Use master's-level technical depth but explain the reasoning clearly. Do not rewrite the Medium article yet. First complete the audit and wait for my next prompt.

CHATGPT RESPONSE:

Evidence-based CRISP-DM audit

Audit basis and evidence hierarchy

I examined:

- The executed notebook
- results-summary.json
- model-comparison.csv
- classification-report.csv
- confusion-matrices.png
- gradcam-explanations.png

Labels used below:

- **Measured evidence:** directly present in an executed output or exported artifact.
- **Inference:** a defensible interpretation of that evidence.
- **Recommendation:** a proposed improvement that has not yet been tested.

Important artifact inconsistency

The confusion counts hard-coded in the source of notebook execution cell `[18]` are:

- Mountain → glacier: 72
- Glacier → mountain: 52
- Street → buildings: 50

But the exported JSON and confusion-matrix image report:

- Mountain → glacier: 69
- Glacier → mountain: 63
- Street → buildings: 49

The latter values should be used because they are read directly from the displayed 6×6 confusion matrix and agree with:

- Its normalized percentages
- The class supports
- The class-level recall values
- The total of 2,708 correct predictions
- The exact accuracy 2708/3000=0.90266672708/3000=0.9026667

This indicates that notebook cell `[18]` was edited after—or was not the exact source used for—the exported JSON and matrix. It weakens artifact lineage, even though the main performance metrics remain internally consistent.

---

1. Business Understanding

What was done correctly

- **Measured evidence:** The notebook defines a concrete six-class scene-classification problem: buildings, forest, glacier, mountain, sea, and street. This appears in execution cells `[2]` and `[5]`, and in `results-summary.json`.
- **Measured evidence:** Two model-development strategies are compared: a custom CNN and ImageNet-initialized MobileNetV2, with test accuracy and categorical cross-entropy loss reported in execution cell `[14]`.
- **Inference:** The experiment has a recognizable technical objective: determine whether transfer learning improves scene classification relative to a from-scratch baseline.

Weaknesses and risks

- No real deployment use case is defined. The notebook does not specify whether this is intended for photo organization, geographic analysis, content moderation, education, or another setting.
- No operational success criterion is declared before training. For example, there is no target accuracy, maximum glacier–mountain error rate, latency limit, or abstention requirement.
- All mistakes are implicitly assigned equal cost. In practice, confusing glacier with mountain may have a different consequence from confusing street with buildings.
- The six labels are not necessarily mutually exclusive at the image level. An image can contain mountains, glaciers, water, buildings, and streets simultaneously.
- There is no discussion of human review, confidence thresholds, or inappropriate uses.

Recommendations

- Define an intended user and decision supported by the prediction.
- Establish pre-training acceptance criteria, including macro-F1, worst-class recall, calibration, latency, and model size.
- Create a cost matrix for operationally important confusions.
- Specify whether the task is “dominant scene classification” rather than assuming each image has one objectively correct semantic label.
- Define an abstention policy for ambiguous or out-of-distribution images.

---

2. Data Understanding

What was done correctly

- **Measured evidence:** Execution cell `[2]` audits the directory structure, file extensions, class names, and counts.
- **Measured evidence:** The original labeled training directory contains 14,034 images:

Class	Training-directory images	Test images
Buildings	2,191	437
Forest	2,271	474
Glacier	2,404	553
Mountain	2,512	525
Sea	2,274	510
Street	2,382	501
Total	14,034	3,000

- **Measured evidence:** Execution cells `[3]` and `[4]` visualize class counts and one example from each class.
- **Inference:** Class imbalance is modest, not extreme. This is consistent with the small difference between MobileNetV2’s accuracy, 90.27%, and macro-F1, 90.48%.
- **Measured evidence:** The unlabeled `seg_pred` directory is detected but not used for evaluation, avoiding unsupported pseudo-ground-truth claims.

Weaknesses and risks

- Only one deterministic example per class is shown: the lexicographically first file in execution cell `[4]`. This cannot characterize intra-class diversity, label ambiguity, corruption, or image quality.
- There is no audit for:
    - Exact duplicates
    - Near-duplicates
    - Cross-split duplicates
    - Corrupted or truncated images
    - Resolution and aspect-ratio distributions
    - Color-mode anomalies
    - Watermarks, borders, or dataset-specific shortcuts
- The train and test sets come from the same packaged dataset. Performance under geographic, seasonal, camera, weather, or web-source shift is unknown.
- No annotation-quality study is conducted. The Grad-CAM examples themselves show semantically mixed scenes, such as mountains with water and glacier-like terrain.
- The notebook does not identify dataset license, provenance limitations, or demographic/geographic coverage.

Leakage risk

- **Measured evidence:** The labeled test directory is kept separate and loaded with `shuffle=False` in execution cell `[5]`.
- **Inference:** There is no direct evidence that test labels entered model training.
- However, duplicate or near-duplicate leakage between `seg_train` and `seg_test` was not tested. Therefore, independence cannot be confirmed.
- Repeated human inspection of test errors can also turn the test set into an informal development set if further model changes are made.

Recommendations

- Compute SHA-256 hashes and perceptual hashes across all splits.
- Sample at least 25–50 images per class, including random, low-confidence, and disputed examples.
- Audit image dimensions, channels, compression, brightness, and corruption.
- Conduct a blinded label review of a stratified sample and report inter-rater agreement.
- Reserve a second, untouched external evaluation set for final claims.

---

3. Data Preparation

What was done correctly

- **Measured evidence:** Execution cell `[5]` creates:
  - 11,228 training images
  - 2,806 validation images
  - 3,000 test images
- **Measured evidence:** Both training and validation use the same seed and `validation_split=0.20`, preventing overlapping subsets under the Keras splitting mechanism.
- **Measured evidence:** The test dataset uses `shuffle=False`, which is necessary for matching predictions to labels.
- **Measured evidence:** Execution cell `[7]` applies augmentation inside the model, so augmentation runs during training and is disabled for evaluation.
- **Measured evidence:** Augmentations include horizontal flipping, rotation, zoom, and contrast.
- **Measured evidence:** Preprocessing is architecture-appropriate:
  - Custom CNN: rescaling to [0,1][0,1], execution cell `[8]`
  - MobileNetV2: `mobilenet_v2.preprocess_input`, execution cell `[11]`

Weaknesses and risks

- The cell `[5]` comment calls the split “stratified-by-folder,” but no explicit stratified split indices or per-split class counts are saved. Keras constructs the split reproducibly, but the artifact does not demonstrate exact class stratification.
- Resizing every image directly to 150×150 can distort aspect ratios and spatial geometry.
- Augmentation realism is not evaluated. Rotations, zoom, and contrast may help, but the notebook does not test whether their ranges preserve label semantics.
- No class-conditional augmentation or hard-example sampling is used.
- No duplicate detection occurs before splitting.
- File lists or split manifests are not exported, making exact sample-level reconstruction difficult.

Reproducibility and nondeterminism

- **Measured evidence:** Python, NumPy, and TensorFlow seeds are set to 42 in execution cell `[2]`.
- **Measured evidence:** TensorFlow 2.20.0 and two Tesla T4 GPUs under `MirroredStrategy` are recorded.

- **Inference:** This improves reproducibility but does not guarantee bitwise repeatability.
- TensorFlow deterministic operations are not enabled.
- Multi-GPU collective reductions, GPU kernels, `AUTOTUNE`, random augmentation, and dataset reshuffling can introduce nondeterminism.
- No second run or seed-sensitivity study is supplied.

Recommendations

- Save explicit train/validation/test manifests containing paths, labels, hashes, and split membership.
- Use padded resize or resize-and-crop and compare it against direct warping.
- Enable deterministic TensorFlow operations where supported.
- Run at least three to five seeds and report mean, standard deviation, and confidence intervals.
- Record package versions, CUDA/cuDNN versions, hardware, dataset version, and model-file checksums.

---

4. Modeling

Custom CNN

What was done correctly

- **Measured evidence:** Execution cell `[8]` defines four convolutional blocks with 32, 64, 128, and 256 filters, Batch Normalization, pooling, global average pooling, a 128-unit dense layer, and 40% dropout.
- **Measured evidence:** The custom model has 424,006 parameters, of which 423,046 are trainable.
- **Measured evidence:** Adam, categorical cross-entropy, augmentation, early stopping, learning-rate reduction, and checkpointing are used.
- Global average pooling limits the parameter count relative to flattening large feature maps.

Weaknesses

- The architecture was apparently evaluated only once. No controlled hyperparameter selection procedure is documented.
- No residual connections, depthwise convolutions, label smoothing, or weight decay are evaluated.

- Execution cell [10] shows:
    - 13 epochs completed
    - Best training accuracy: 88.37%
    - Best validation accuracy: 80.29%
    - Lowest validation loss: 0.5437
- **Inference:** The roughly eight-percentage-point training–validation accuracy gap suggests overfitting or train/validation distribution differences. It is not proof of overfitting by itself because training uses stochastic augmentation while validation does not.
- Checkpointing monitors validation accuracy, while early stopping restores the minimum-validation-loss state. Different selection criteria can identify different epochs.

MobileNetV2

What was done correctly

- **Measured evidence:** Execution cell [11] initializes MobileNetV2 from ImageNet, initially freezes its convolutional base, uses global average pooling and 35% dropout, and applies correct MobileNetV2 preprocessing.
- **Measured evidence:** Execution cell [13] unfreezes only the final 30 layers, freezes Batch Normalization layers, lowers the learning rate to 10−510^{-5}, and reports 1,518,406 trainable parameters.
- Freezing Batch Normalization is defensible for relatively small datasets and limited effective batch statistics.
- Staged transfer learning reduces the risk of immediately destroying pretrained representations.

Weaknesses

- MobileNetV2 emits a warning that 224×224 pretrained weights are loaded while the model input is 150×150. This is supported by execution cell [11].
- These convolutional weights can operate on 150×150 inputs, so this is not necessarily an error. However, the effect of reduced resolution is not evaluated.
- Only one pretrained backbone is studied.
- The fine-tuning boundary of “last 30 layers” is not justified experimentally.
- There is no frozen-transfer versus fine-tuned test comparison, so the incremental benefit of fine-tuning cannot be quantified.
- The comparison is not controlled for optimization budget, trainable parameter count, augmentation exposure, or initialization.

Checkpoint-selection inconsistency

Execution cell `[14]` loads `best-custom-cnn.keras` for the custom model but evaluates the in-memory `transfer_model` for MobileNetV2. The saved `best-fine-tuned-model.keras` is not reloaded.

Early stopping restored the MobileNetV2 weights from epoch 3, which was both the lowest observed validation-loss epoch and highest validation-accuracy epoch in the fine-tuning output, so the in-memory model likely corresponds to that epoch. Nevertheless, the actual saved checkpoint was not loaded and checked. Model identity should be verified by checksum or prediction equality.

Why transfer learning performed better

- **Measured evidence:** MobileNetV2 achieved 90.27% test accuracy versus 80.10% for the custom CNN, a difference of 10.17 percentage points. Its test loss was also lower: 0.2487 versus 0.5698.
- **Inference:** The most plausible explanation is that ImageNet pretraining supplied reusable visual representations for edges, textures, shapes, and object/scene components. Those representations were then adapted using conservative fine-tuning.
- Appropriate preprocessing, depthwise-separable architecture, staged optimization, dropout, frozen Batch Normalization, and pretrained initialization may all contribute.
- The evidence does **not** establish that greater complexity alone caused the improvement. In fact, multiple factors differ simultaneously, and no ablation isolates their effects.

---

5. Evaluation

What was done correctly

- **Measured evidence:** The untouched ordered test set contains 3,000 images.
- **Measured evidence:** Accuracy and categorical cross-entropy loss are reported for both models.
- **Measured evidence:** MobileNetV2 additionally receives class-level precision, recall, F1, macro averages, weighted averages, a confusion matrix, confident-error inspection, and Grad-CAM analysis.
- **Measured evidence:** All MobileNetV2 class supports and confusion-matrix row totals agree exactly.
- **Measured evidence:** The confusion-matrix diagonal contains 2,708 correct classifications, yielding the reported 90.2667% accuracy.

Class-level MobileNetV2 results

Class	Precision	Recall	F1	Support
Buildings	0.8849	0.9497	0.9161	437
Forest	0.9874	0.9916	0.9895	474
Glacier	0.8423	0.8210	0.8315	553
Mountain	0.8608	0.8248	0.8424	525
Sea	0.9072	0.9588	0.9323	510
Street	0.9430	0.8922	0.9169	501
Macro average	0.9043	0.9063	0.9048	3,000
Weighted average	0.9025	0.9027	0.9021	3,000

Source: execution cell `[14]` and `classification-report.csv`.

Confusion patterns

The dominant errors are:

Actual → predicted	Count	Row percentage
Mountain → glacier	69	13.1%
Glacier → mountain	63	11.4%
Street → buildings	49	9.8%
Glacier → sea	30	5.4%
Buildings → street	20	4.6%

- **Inference:** Glacier and mountain are the hardest semantic pair. Their lower recall and F1 values support this.
- **Inference:** Street/buildings errors likely arise from urban co-occurrence and label ambiguity rather than purely random failure.
- Forest is exceptionally separable in this test set: 470 of 474 images are correct.

Confident mistakes and calibration

The Grad-CAM figure displays six of the most confident errors, with reported confidence from 98.5% to 99.8%:

- Street → buildings: 99.8%
- Glacier → sea: 99.6%
- Mountain → sea: 99.3%
- Glacier → mountain: 98.9%
- Glacier → sea: 98.6%
- Mountain → glacier: 98.5%
- **Measured evidence:** These are high-confidence wrong predictions.
- **Inference:** They demonstrate that maximum softmax probability cannot safely be interpreted as correctness probability.
- They do not, by themselves, prove global miscalibration because no expected calibration error, reliability diagram, Brier score, or negative log-likelihood comparison is supplied.
- The selection is intentionally biased toward extreme errors, so it cannot represent typical mistakes.

Grad-CAM

Supported observations

- For the street → buildings error, activation is concentrated on prominent architecture, which is consistent with the predicted class.
- For sea-related errors, activation often follows water or horizon regions.
- For mountain/glacier errors, activation overlaps rock, snow-like textures, and terrain boundaries.
- Some heatmaps appear broad and diffuse, suggesting reliance on global texture and composition.

Limitations

- Only six selected errors are shown.
- No correct predictions, low-confidence cases, or random baseline cases are included.
- Grad-CAM is qualitative and low resolution.
- It does not establish causal feature use or model fairness.
- The use of the post-softmax class score can produce weaker gradients than using the pre-softmax logit.
- No sanity checks, such as parameter randomization or explanation stability under perturbations, are performed.

Missing evaluation

- Custom CNN class-level metrics and confusion matrix
- Per-model predictions
- Confidence intervals
- Repeated-run variability
- Statistical comparison on paired predictions, such as McNemar's test
- ROC/PR analyses where operationally meaningful
- Calibration metrics
- Top-2 accuracy
- Selective-classification coverage versus risk
- Robustness to blur, brightness, crop, compression, and corruption
- External-distribution evaluation
- Latency, memory, throughput, and model size comparison

---

6. Deployment

What was done correctly

- **Measured evidence:** Execution cell `[18]` saves the final fine-tuned model in `.keras` format.
- Machine-readable JSON and CSV results are exported.
- The class order and image size are recorded in `results-summary.json`.
- The final model includes preprocessing and augmentation layers, although augmentation should be inactive during inference.

Weaknesses

- No inference function or serving contract is supplied.
- No class-index metadata is packaged directly with a deployable interface.
- No model signature, checksum, version, or dataset fingerprint is reported.
- No latency, throughput, CPU inference, memory, quantization, or conversion tests are provided.
- No confidence-calibration or abstention rule exists.
- No monitoring or retraining policy is defined.
- Input validation and out-of-distribution handling are absent.

Verified results table

Result	Custom CNN	Fine-tuned MobileNetV2	Verification status
Test images	3,000	3,000	Verified
Test accuracy	0.8010	0.9027	Verified across notebook, CSV, JSON
Correct predictions	Not exported	2,708	Verified from confusion matrix
Test loss	0.5698	0.2487	Verified across notebook, CSV, JSON
Macro precision	Not computed	0.9043	Verified
Macro recall	Not computed	0.9063	Verified
Macro F1	Not computed	0.9048	Verified
Weighted F1	Not computed	0.9021	Verified
Accuracy gain	—	+0.1017 absolute	Derived from measured accuracies
Error reduction	—	51.1% relative	Derived: (0.1990−0.09733)/0.1990(0.1990-0.09733)/0.1990
Evaluation model identity	Best accuracy checkpoint loaded	Restored in-memory model	MobileNet checkpoint not explicitly reloaded

The relative error reduction is a derived comparison, not an independently measured notebook metric.

Five principal findings

1. MobileNetV2 materially outperformed the custom CNN on both accuracy and loss: 90.27% versus 80.10%, and 0.2487 versus 0.5698.
2. Performance is reasonably balanced across classes, but glacier and mountain remain substantially harder than forest, sea, buildings, and street.
3. The model makes extremely confident errors, so raw softmax confidence is unsuitable as a reliability measure without calibration.
4. Grad-CAM qualitatively indicates reliance on semantically plausible regions, but the limited, selected examples cannot validate general explanation faithfulness.

5. Artifact lineage is imperfect: notebook cell `[18]` contains confusion counts that disagree with the exported matrix and JSON, and the saved MobileNet checkpoint is not explicitly reloaded for testing.

Five limitations

1. No custom-CNN class-level metrics, confusion matrix, predictions, or error analysis are available.
2. Results come from one split and one run; uncertainty and seed sensitivity are unknown.
3. Duplicate leakage, label quality, and near-duplicate independence were not audited.
4. Calibration, robustness, external distribution shift, and out-of-distribution behavior were not measured.
5. The experiment does not isolate why transfer learning won because architecture, initialization, preprocessing, trainable parameters, and optimization differ together.

Five technically feasible improvements

1. Export sample-level predictions for both models and run paired comparison, confidence intervals, McNemar's test, and complete per-class evaluation.
2. Add hash/perceptual-hash duplicate detection, corruption checks, explicit split manifests, and a manual ambiguity audit.
3. Run multiple seeds and controlled ablations: random-initialized MobileNetV2, frozen MobileNetV2, fine-tuned MobileNetV2, and matched augmentation settings.
4. Measure calibration with reliability diagrams, ECE, Brier score, and NLL; fit temperature scaling using validation data only.
5. Evaluate robustness and deployment characteristics across resolutions, corruptions, external images, CPU/GPU latency, memory, and quantized formats.

Deployment and monitoring plan

1. **Package**
   Save the selected checkpoint, class mapping, preprocessing contract, TensorFlow version, input shape, dataset hash, git/notebook version, and model checksum.
2. **Validate before release**
   Reload the saved artifact in a clean environment and confirm prediction equality against stored reference inputs. Evaluate latency, memory, batch throughput, and corrupted-input behavior.

3. **Calibrate and abstain**
   Fit temperature scaling on validation predictions. Choose a validation-based acceptance threshold; route uncertain or out-of-distribution images to human review.
4. **Log safely**
   Record model version, predicted probabilities, abstention outcome, input-quality statistics, latency, and—when later available—verified labels. Avoid retaining sensitive raw images unnecessarily.
5. **Monitor**
   Track:
   - Class-frequency drift
   - Input embedding drift
   - Confidence and entropy drift
   - Per-class precision/recall on reviewed samples
   - Calibration error
   - Abstention rate
   - Glacier–mountain and street–buildings confusion rates
   - Latency and failure rates
6. **Respond**
   Define alert thresholds relative to the validated baseline. Investigate data-pipeline changes before retraining.
7. **Retrain and promote**
   Version new data, preserve a locked regression test set, compare challenger and production models with paired tests, and require explicit approval before promotion. Maintain rollback capability.

Reproducibility checklist

- Random seed recorded: 42
- TensorFlow version recorded: 2.20.0
- GPU type and count visible: two Tesla T4 GPUs
- Image size recorded: 150×150
- Batch size recorded: 64
- Class ordering recorded
- Train/validation/test counts recorded
- Main architectures and callbacks preserved
- Final metrics exported to CSV/JSON
- Exact dataset version or checksum recorded
- File-level split manifests exported
- Package/CUDA/cuDNN environment locked
- Deterministic TensorFlow operations enabled
- Multiple seeds executed
- Saved-model checksums recorded
- Best MobileNet checkpoint explicitly reloaded for evaluation

- Sample-level predictions exported
- Artifact-generation lineage verified
- Conflicting hard-coded confusion counts corrected
- Clean-environment inference test performed

Questions that cannot be answered from the artifacts

1. Does MobileNetV2's advantage remain statistically reliable across different seeds and data splits?
2. What are the custom CNN's class-level precision, recall, F1, confusion patterns, and confident mistakes?
3. Are there exact or near-duplicate images across training, validation, and test sets?
4. How many images have ambiguous or incorrect dominant-scene labels?
5. Is the model calibrated over the full test set?
6. How does performance change under geographic, seasonal, camera, weather, or compression shift?
7. What test score is obtained by explicitly reloading `best-fine-tuned-model.keras`?
8. Do the saved checkpoint and final in-memory model produce identical predictions?
9. What caused the inconsistent confusion counts in notebook cell `[18]`?
10. How fast and memory-efficient is the model on the intended deployment hardware?
11. How stable and faithful are the Grad-CAM explanations?
12. What performance threshold and error costs would make this model suitable for its intended real-world use?

The audit is complete. I have not rewritten the Medium article.

PROMPT:

Critically review your previous audit as if you were a skeptical peer reviewer.

Identify:

1. Any claim not directly supported by an attached artifact
2. Any place where association may have been presented as causation
3. Any missing reproducibility details
4. Any weaknesses in the train-validation-test methodology
5. Any metric that needs additional context
6. Any conclusion that should be softened or removed

Return a claim-audit table with these columns:

- Original claim
- Evidence
- Classification: measured, inferred, or unsupported
- Required correction

Then provide a corrected final audit. Do not invent new experimental results.

CHATGPT RESPONSE:

Skeptical peer review of the previous audit

The previous audit was broadly evidence-aligned, but several statements need tighter qualification. Its main weakness was occasionally treating standard TensorFlow behavior or plausible interpretations as though they had been empirically verified in this particular run.

Claim-audit table

Original claim	Evidence	Classification	Required correction
“The latter confusion values should be used.”	The displayed matrix reports 69 mountain→glacier, 63 glacier→mountain, and 49 street→buildings. These counts reproduce the exported recalls and overall accuracy. Cell [18] source contains 72, 52, and 50.	Inferred	State that 69/63/49 are the most internally consistent values, not necessarily the historically authoritative values. Raw predictions are unavailable, so their provenance cannot be independently reconstructed.

“This indicates that notebook cell `[18]` was edited after—or was not the exact source used for—the exported JSON and matrix.”	Cell `[18]` source conflicts with the attached JSON and matrix.	Inferred	Soften to: “The artifacts were not generated from one demonstrably consistent source state.” The precise cause and chronology are unknown.
“There is no direct evidence that test labels entered model training.”	Training uses `TRAIN_DIR`; evaluation uses `TEST_DIR`.	Measured	Retain, but add that later human inspection of test errors may influence subsequent development; the artifacts do not show whether that occurred.
“Both training and validation use the same seed and `validation_split=0.20`, preventing overlapping subsets.”	Cell `[5]` uses complementary `subset="training"` and `subset="validation"` with the same seed and split fraction.	Inferred	Say this is intended to create complementary subsets under Keras behavior. File-level manifests are absent, so overlap was not independently verified.

“The split is reproducible.”	Seed 42 is supplied to both dataset constructors.	Inferred	Replace with “the split is designed to be repeatable under the same TensorFlow version, directory contents, and file ordering.”
“The cell comment calls the split stratified-by-folder, but no explicit stratified indices are saved.”	Cell `[5]` contains that comment, but no per-class split table or manifest.	Measured	Strengthen: explicit stratification is not demonstrated. Do not describe the split as stratified without per-class split evidence.
“Augmentation runs during training and is disabled for evaluation.”	Augmentation is a Keras preprocessing model embedded before the classifiers; evaluation calls do not specify `training=True`.	Inferred	State this is expected Keras behavior, but the artifacts contain no direct augmented-versus-unaugmented evaluation check.

“The roughly eight-percentage-point training–validation gap suggests overfitting.”	Cell `[10]` gives best training accuracy 0.8837 and best validation accuracy 0.8029.	Weak inference	Soften substantially. These are maxima that may occur at different epochs, and training uses augmentation while validation does not. Use epoch-aligned learning curves before diagnosing overfitting.
“Class imbalance is modest, not extreme.”	Class counts range from 2,191 to 2,512 in the labeled training directory and 437 to 553 in test.	Inferred	Retain as a descriptive statement, but quantify it: largest/smallest ratios are approximately 1.15 for the training directory and 1.27 for test. Avoid implying imbalance has no effect.
“This is consistent with the small difference between accuracy and macro-F1.”	Accuracy is 0.9027 and macro-F1 is 0.9048.	Inferred	Retain only as consistency, not proof that imbalance is harmless. Macro-F1 and accuracy capture different quantities.

“The experiment has a recognizable technical objective.”	Two classifiers are trained and compared.	Inferred	State that the technical comparison is apparent, but the notebook does not formally define a business objective, decision context, or acceptance threshold.
“The most plausible explanation is that ImageNet pretraining supplied reusable visual representations.”	MobileNetV2 uses ImageNet weights and performs better. No initialization ablation exists.	Inferred	Make the non-causal status prominent. Say pretraining is a plausible contributor, but its effect cannot be separated from architecture, preprocessing, parameter count, and training schedule.
“Staged transfer learning reduces the risk of immediately destroying pretrained representations.”	Base is frozen initially and later partially unfrozen at a lower learning rate.	General methodological inference	Rephrase as the intended purpose of the procedure, not a measured result. No comparison with immediate full fine-tuning exists.

“Freezing Batch Normalization is defensible for relatively small datasets.”	BN layers are explicitly frozen in cell [13].	Recommendation/judgment	Say only that freezing BN is a conventional design choice. Its benefit in this experiment was not measured.
“The MobileNetV2 in-memory model likely corresponds to the best checkpoint.”	Fine-tuning epoch 3 has the best displayed validation accuracy and validation loss; early stopping restores epoch 3.	Inferred	Retain with qualification. The saved checkpoint was not reloaded and prediction equivalence was not verified.
“MobileNetV2 materially outperformed the custom CNN.”	Test accuracies are 0.9027 and 0.8010; losses are 0.2487 and 0.5698.	Measured descriptively	Replace “materially” with “achieved higher accuracy and lower loss in this single run.” Statistical and practical significance were not established.
“Relative error reduction was 51.1%.”	Derived from the two observed error rates.	Measured derivation	Retain only with context: it is a single-run descriptive calculation, highly sensitive to the chosen baseline, and not a confidence-bound effect estimate.

“Forest is exceptionally separable.”	Forest recall is 0.9916; 470/474 forest images are correct.	Inferred	Replace with “forest had the highest recall on this test set.” Separability outside this dataset is not established.
“The high-confidence errors demonstrate that maximum softmax probability cannot safely be interpreted as correctness probability.”	Six selected wrong predictions have confidence 98.5–99.8%.	Inferred	Retain as a warning about these cases, but do not infer population-wide calibration failure. The sample was selected specifically for high-confidence errors.
“The Grad-CAM maps indicate reliance on semantically plausible regions.”	The supplied figure visually overlaps architecture, water, and terrain in selected examples.	Inferred	Qualify as subjective visual interpretation. Grad-CAM does not establish causal reliance, explanation faithfulness, or correct reasoning.
“The model includes preprocessing and augmentation layers.”	Cell [11] places both before MobileNetV2. Cell [18] saves the in-memory model.	Measured from code	Retain for the notebook model definition, but checkpoint contents were

			not separately inspected.
“Performance under distribution shift is unknown.”	No external or shifted evaluation appears in the artifacts.	Measured absence	Retain.
“No corrupted-image audit was conducted.”	No such code or table appears in the notebook.	Measured absence	Retain, limited to the supplied artifacts.
“No inference function or serving contract is supplied.”	The artifacts contain training/evaluation code and model exports, but no serving interface.	Measured absence	Retain.
“The public test set was untouched.”	The test data are not passed to `fit`, but are later used for metrics, error selection, and Grad-CAM.	Unsupported if stated absolutely	Replace with “the test set was not used by the displayed training calls.” It was used extensively after training, and future tuning based on this analysis would compromise its role as a final test set.

Corrected final audit

Scope and evidentiary rules

This audit treats the executed notebook, exported JSON/CSV files, and supplied figures as the evidence. It does not assume that the reported run is repeatable, statistically representative, calibrated, or deployment-ready.

Three levels are used:

- **Measured:** directly visible in an attached artifact.
- **Inference:** interpretation consistent with the evidence but not experimentally isolated.
- **Recommendation:** an untested proposed improvement.

Artifact consistency

The headline metrics agree across execution cell `[14]`, `model-comparison.csv`, and `results-summary.json`:

- Custom CNN: test accuracy 0.8010, loss 0.5698
- Fine-tuned MobileNetV2: test accuracy 0.9027, loss 0.2487
- MobileNetV2 macro precision 0.9043, macro recall 0.9063, macro-F1 0.9048

The confusion-matrix figure is internally consistent with `classification-report.csv`:

- Diagonal total: 2,708
- Test total: 3,000
- Accuracy: 2708/3000=0.90266672708/3000=0.9026667

However, notebook cell `[18]` contains hard-coded confusion counts of 72, 52, and 50, while the attached JSON and matrix contain 69, 63, and 49. The cause cannot be determined from the artifacts.

For this audit, 69/63/49 are used because they are mutually consistent with the displayed matrix, normalized percentages, class supports, recall values, and overall accuracy. This resolves the numerical analysis but not the provenance problem.

---

1. Business Understanding

Correct

- **Measured:** The project performs six-class scene classification across buildings, forest, glacier, mountain, sea, and street.
- **Measured:** It compares a custom CNN with ImageNet-initialized MobileNetV2.
- **Inference:** The apparent technical objective is to compare from-scratch learning with transfer learning.

Weaknesses

The notebook does not define:

- A real user or deployment scenario
- The decision supported by a prediction
- A predeclared success threshold
- Error costs
- Latency or memory constraints
- Human-review requirements
- The meaning of a label for mixed-content images

The classes are potentially semantically overlapping. A photograph can contain a lake, mountain, glacier, road, and buildings while receiving only one folder label. The artifacts do not establish whether labels represent the dominant scene, photographer intent, or another annotation policy.

Risks

- Accuracy may not correspond to operational value.
- A model can meet aggregate accuracy while failing an important class or use case.
- Some reported “errors” may reflect ambiguous single-label annotation rather than unequivocal visual failure.

Recommendation

Define the intended decision, label policy, cost matrix, acceptance thresholds, and abstention/human-review process before further model selection.

---

2. Data Understanding

Correct

- **Measured:** Cell `[2]` audits directory existence, supported file extensions, class names, and counts.
- **Measured:** The labeled training directory has 14,034 images; the test directory has 3,000.
- **Measured:** Class distributions are:

Class	Labeled training directory	Test

Buildings	2,191	437
Forest	2,271	474
Glacier	2,404	553
Mountain	2,512	525
Sea	2,274	510
Street	2,382	501

- **Measured:** Cell `[4]` displays one image per class.
- **Measured:** The unlabeled prediction directory is discovered but not assigned artificial evaluation labels.

Context

The largest-to-smallest class-count ratio is approximately:

- 1.15 in the labeled training directory
- 1.27 in the test directory

This is limited imbalance, but it does not show that imbalance has no effect. Macro and per-class metrics remain necessary.

Weaknesses

One lexicographically selected example per class does not adequately audit:

- Intra-class variation
- Mixed-scene ambiguity
- Label errors
- Corrupted images
- Image-size variation
- Watermarks or borders
- Exact duplicates
- Near-duplicates
- Cross-split overlap

Dataset provenance, version, license, geographic composition, and image-source distribution are not recorded in the exported results.

Leakage and distribution-shift risk

- **Measured:** The displayed `fit` calls use the training directory, not the test directory.
- **Unknown:** Whether train and test contain duplicate or near-duplicate scenes.
- **Unknown:** Whether test images are independent by source, photographer, location, or sequence.
- **Unknown:** Whether the model generalizes beyond this dataset's acquisition distribution.

Recommendation

Create an image inventory containing path, class, dimensions, color mode, cryptographic hash, perceptual hash, split, and corruption status. Add a blinded human audit of random and disagreement-heavy cases.

---

3. Data Preparation

Correct

- **Measured:** Cell `[5]` reports 11,228 training, 2,806 validation, and 3,000 test images.
- **Measured:** Training and validation are generated from the same 14,034-image directory using `validation_split=0.20`, complementary subset names, and seed 42.
- **Measured:** Test loading uses `shuffle=False`.
- **Measured:** Training-time transformations include horizontal flip, rotation, zoom, and contrast.
- **Measured:** The custom CNN rescales pixels by 1/2551/255; MobileNetV2 uses its application-specific preprocessing function.

Methodological weaknesses

1. **Stratification is not demonstrated.**
   Cell `[5]` describes the split as "stratified-by-folder," but no per-class training/validation counts or explicit stratified indices are exported.
2. **Split membership is not preserved.**
   Without file manifests, an independent reviewer cannot verify membership, overlap, or exact reconstruction.
3. **Potential duplicate leakage remains untested.**
4. **Direct 150×150 resizing may alter aspect ratio.**
   The artifacts contain no comparison with padded resizing or crop-based preprocessing.
5. **Augmentation was not validated through ablation.**
   The visualization in cell `[7]` shows that transformations occur, not that they improve generalization.

Reproducibility

Seeds are set for Python, NumPy, and TensorFlow, but exact repeatability is not established.

Missing details include:

- Deterministic-operation setting
- CUDA and cuDNN versions
- Complete package environment
- Dataset checksums
- File ordering and split manifests
- Random-augmentation seed behavior
- Model-file checksums
- Repeat runs demonstrating variance

`MirroredStrategy`, GPU reductions, data prefetching, and some TensorFlow kernels may produce nondeterminism. No repeated run tests this.

Recommendation

Export exact split manifests and environment metadata; enable deterministic execution where feasible; test several seeds; and explicitly audit split overlap.

---

4. Modeling

Custom CNN

- **Measured:** Four convolutional blocks, Batch Normalization, max pooling, global average pooling, a 128-unit dense layer, 40% dropout, and a six-class softmax output are used.
- **Measured:** Total parameters: 424,006; trainable parameters: 423,046.
- **Measured:** Training uses Adam at 10−310^{-3}, categorical cross-entropy, early stopping, learning-rate reduction, and validation-accuracy checkpointing.
- **Measured:** Cell `[10]` reports 13 epochs, maximum training accuracy 0.8837, maximum validation accuracy 0.8029, and minimum validation loss 0.5437.

The previous audit overstated the evidentiary value of the training–validation gap. The two maxima may come from different epochs, and training accuracy is measured while augmentation and dropout are active. Overfitting is possible, but an epoch-aligned analysis is required before making a strong diagnosis.

MobileNetV2

- **Measured:** ImageNet weights are loaded.
- **Measured:** The feature extractor is initially frozen.

- **Measured:** The classifier uses global average pooling, 35% dropout, and a six-class softmax output.
- **Measured:** The last 30 base-model layers are made eligible for fine-tuning, while Batch Normalization layers remain frozen.
- **Measured:** Fine-tuning uses Adam at 10−510^{-5} with 1,518,406 trainable parameters.
- **Measured:** Fine-tuning stops after epoch 5 and restores epoch 3.
- **Measured:** Epoch 3 has displayed validation accuracy 0.91055 and validation loss 0.2511.

The warning about loading 224×224-derived ImageNet weights for a 150×150 input is recorded in cell `[11]`. The artifacts do not establish whether 150×150 is better or worse than another resolution.

Fairness of the comparison

The experiment changes multiple factors simultaneously:

- Architecture
- ImageNet initialization
- Preprocessing
- Trainable parameter count
- Optimization schedule
- Freeze/unfreeze behavior
- Dropout rate
- Training duration

Therefore, the experiment compares two complete pipelines. It does not isolate the causal effect of pretraining, fine-tuning, architecture, or parameter count.

Checkpoint methodology

The custom model is reloaded from `best-custom-cnn.keras`.

MobileNetV2 is evaluated from the in-memory model after early stopping restoration; `best-fine-tuned-model.keras` is not reloaded. Because both checkpointing and early stopping selected epoch 3 in the displayed output, these models may be equivalent, but equivalence is not verified.

Recommendation

Evaluate four controlled conditions:

1. Custom CNN from random initialization
2. MobileNetV2 from random initialization
3. Frozen ImageNet MobileNetV2

4. Partially fine-tuned ImageNet MobileNetV2

Use matched data splits and repeated seeds. Explicitly reload every chosen checkpoint before final evaluation.

---

5. Evaluation

Verified model-level results

Model	Test loss	Test accuracy	Macro precision	Macro recall	Macro-F1
Custom CNN	0.5698	0.8010	Not computed	Not computed	Not computed
Fine-tuned MobileNetV2	0.2487	0.9027	0.9043	0.9063	0.9048

MobileNetV2 achieved 10.17 percentage points higher test accuracy in this run. This is a descriptive difference, not a statistical confidence statement.

The derived relative error reduction is approximately 51.1%, but it should be interpreted cautiously because:

- It comes from one run
- It depends on the custom CNN as the baseline
- No uncertainty interval is available

MobileNetV2 class-level results

Class	Precision	Recall	F1	Support
Buildings	0.8849	0.9497	0.9161	437
Forest	0.9874	0.9916	0.9895	474
Glacier	0.8423	0.8210	0.8315	553
Mountain	0.8608	0.8248	0.8424	525
Sea	0.9072	0.9588	0.9323	510
Street	0.9430	0.8922	0.9169	501

Macro average	0.9043	0.9063	0.9048	3,000
Weighted average	0.9025	0.9027	0.9021	3,000

Forest has the highest test-set recall and glacier has the lowest. This ranking is specific to the supplied test set.

Confusion patterns

Actual → predicted	Count	Percentage of actual class
Mountain → glacier	69	13.1%
Glacier → mountain	63	11.4%
Street → buildings	49	9.8%
Glacier → sea	30	5.4%
Buildings → street	20	4.6%

Glacier–mountain is the dominant reciprocal confusion in the displayed matrix. This may reflect visual similarity, annotation ambiguity, model limitations, or a combination; the artifacts cannot separate those explanations.

Loss context

Both models use categorical cross-entropy on the same test set, so their test losses are directly comparable within this experiment. Loss captures probability assignment, not just the winning class, but it is not itself a calibrated probability metric.

Confident errors

The supplied Grad-CAM figure shows six deliberately selected high-confidence errors with maximum softmax scores from 98.5% to 99.8%.

This establishes that some wrong predictions receive very high softmax scores. It does not establish the model's overall calibration because:

- The cases were selected for being the most confident errors
- Correct-case confidence is not shown
- No reliability diagram, ECE, Brier score, or calibrated NLL is reported

Grad-CAM

The heatmaps overlap visually salient architecture, water, horizon, and terrain regions in these six examples. That observation is subjective and qualitative.

Grad-CAM cannot establish that:

- These regions caused the decision
- The model used human-like reasoning
- Explanations are stable
- Explanations are faithful across the test distribution

The analysis also lacks correct examples, random examples, explanation sanity checks, and comparisons across layers or methods.

Evaluation methodology weaknesses

- Custom CNN has no per-class metrics or predictions.
- No repeated runs or confidence intervals are available.
- No paired statistical test can be performed without both models' sample-level predictions.
- The test set is used for detailed error analysis. Any later tuning informed by this analysis would require a new untouched test set.
- No calibration, external validation, corruption testing, or out-of-distribution evaluation is supplied.

---

6. Deployment

Available evidence

- **Measured:** Cell `[18]` saves the final in-memory MobileNetV2 model.
- **Measured:** CSV and JSON summaries are produced.
- **Measured from notebook code:** The model definition includes preprocessing and a class order.

Missing deployment evidence

- Clean-environment model reload
- Prediction-equivalence test
- Model checksum
- Serving signature
- Input validation
- CPU/GPU latency
- Throughput and memory use

- Quantization or conversion testing
- Calibration
- Abstention threshold
- Out-of-distribution detection
- Monitoring and rollback procedures

The artifacts support model export, not deployment readiness.

Five corrected principal findings

1. In this single run, MobileNetV2 achieved higher test accuracy and lower test loss than the custom CNN.
2. Glacier and mountain form the largest reciprocal confusion in the internally consistent exported matrix.
3. Some selected wrong predictions received softmax scores above 98%, warranting calibration analysis but not proving global miscalibration.
4. Grad-CAM provides qualitative visualizations for six selected errors but does not establish causal or faithful explanations.
5. Inconsistent confusion counts and the absence of explicit MobileNet checkpoint reloading weaken provenance and exact reproducibility.

Five limitations

1. Only one execution and one data split are represented.
2. Custom-CNN class-level results and sample-level predictions are absent.
3. Exact and near-duplicate leakage were not assessed.
4. Split manifests, environment lock files, and model checksums are missing.
5. Calibration, robustness, distribution shift, and deployment performance were not evaluated.

Five feasible improvements

1. Export exact split manifests and sample-level probabilities for both models.
2. Audit duplicates, corrupted files, image properties, and ambiguous labels.
3. Repeat training over several seeds and report uncertainty.
4. Run controlled pretraining, freezing, fine-tuning, and resolution ablations.
5. Add calibration, robustness, external-distribution, and serving-performance evaluation.

Deployment and monitoring plan

- Reload the chosen checkpoint in a clean environment and verify predictions against stored reference cases.
- Package the class map, preprocessing contract, model checksum, environment specification, and dataset fingerprint.
- Calibrate probabilities using validation data only and define an abstention threshold.
- Monitor input drift, predicted-class distribution, entropy, abstention rate, latency, and verified per-class performance.
- Pay particular attention to glacier–mountain and street–buildings confusions.
- Require a new locked test set once the current test-error analysis informs further development.
- Maintain versioned rollout and rollback procedures.

Reproducibility checklist

- Python, NumPy, and TensorFlow seed specified
- TensorFlow version reported
- GPU type and count reported
- Image size and batch size specified
- Class order and aggregate split sizes reported
- Architecture and callbacks present
- Main metrics exported
- Dataset version and checksums
- File-level split manifests
- Exact dependency lock file
- CUDA/cuDNN versions
- Deterministic-operation configuration
- Multiple-seed results
- Sample-level predictions for both models
- Saved-model checksums
- Explicit reload of the selected MobileNet checkpoint
- Clean-environment inference verification
- Resolved artifact-lineage inconsistency

Questions that remain unanswered

1. Are any images duplicated or near-duplicated across splits?
2. What are the custom CNN’s per-class metrics and confusion matrix?
3. How variable are the results across seeds?

4. Is the observed accuracy difference statistically reliable under paired testing?
5. What is the isolated contribution of ImageNet pretraining?
6. What is the isolated contribution of fine-tuning?
7. Are labels ambiguous or erroneous, particularly for glacier, mountain, sea, buildings, and street?
8. Is MobileNetV2 calibrated over the complete test distribution?
9. How does it perform under external or corrupted inputs?
10. Does the saved best MobileNet checkpoint exactly reproduce the reported predictions?
11. Why do cell `[18]` and the exported confusion artifacts disagree?
12. What latency, memory, and throughput can be achieved on the intended deployment hardware?

PROMPT:

Focus only on model failure analysis.

Using the classification report, confusion matrix, confident-error gallery, and Grad-CAM figure:

1. Rank the six classes from strongest to weakest.
2. Quantify the largest directional confusions.
3. Explain plausible visual reasons for glacier-mountain and street-buildings confusion.
4. Distinguish model error from possible label ambiguity.
5. Explain what Grad-CAM supports and what it cannot prove.
6. Propose three experiments that could test the explanations.

Separate every conclusion into:

- Observed evidence
- Interpretation
- Proposed validation experiment

CHATGPT RESPONSE:

1. Class ranking

Ranking criterion: class-level F1, because it balances precision and recall.

Rank	Class	Precision	Recall	F1	Key failure characteristic
1	Forest	0.9874	0.9916	0.9895	Only 4 of 474 images misclassified
2	Sea	0.9072	0.9588	0.9323	High recall, but lower precision from glacier/mountain images predicted as sea

3	Street	0.9430	0.8922	0.9169	49 street images predicted as buildings
4	Buildings	0.8849	0.9497	0.9161	High recall but lower precision, partly because street images enter this class
5	Mountain	0.8608	0.8248	0.8424	69 mountain images predicted as glacier
6	Glacier	0.8423	0.8210	0.8315	63 glacier images predicted as mountain and 30 as sea

Observed evidence: Forest has the highest F1 and glacier the lowest. Street narrowly exceeds buildings in F1, although buildings has higher recall.

Interpretation: Forest is the strongest class on this test set. Glacier and mountain are the weakest because of substantial reciprocal confusion. Street and buildings have similar F1 scores but different failure profiles: street has stronger precision, while buildings has stronger recall.

Proposed validation experiment: Repeat the class ranking across multiple seeds and bootstrap the test predictions to obtain confidence intervals for each F1. Without uncertainty estimates, small differences—especially street versus buildings—should not be treated as a stable ordering.

---

2. Largest directional confusions

Rank	Actual class → predicted class	Count	Percentage of actual class
1	Mountain → glacier	69	13.1%
2	Glacier → mountain	63	11.4%
3	Street → buildings	49	9.8%
4	Glacier → sea	30	5.4%
5	Buildings → street	20	4.6%
6	Mountain → sea	16	3.0%
7	Sea → glacier	14	2.7%

Observed evidence: Mountain–glacier confusion is strongly bidirectional: 132 images cross between the two classes. Street–buildings confusion is asymmetric: 49 street images are classified as buildings, while only 20 building images are classified as street.

Interpretation: The mountain–glacier boundary is the model's dominant class-boundary problem. The street–buildings boundary appears biased toward predicting buildings when an urban image contains both road and architectural content.

Proposed validation experiment: Export sample-level predictions and manually categorize every error in these three directions by visible attributes—snow/ice, exposed rock, water, roadway area, façade area, skyline, and mixed-scene composition. Test whether these attributes occur significantly more often in errors than in correctly classified images.

---

3. Glacier–mountain confusion

Shared terrain and texture

Observed evidence: The matrix contains 69 mountain→glacier and 63 glacier→mountain errors. Glacier and mountain have the two lowest recalls, 0.8210 and 0.8248.

The Grad-CAM gallery includes:

- A mountain predicted as glacier at 98.5% confidence
- A glacier predicted as mountain at 98.9% confidence

Their heatmaps overlap prominent terrain regions.

Interpretation: Both classes can contain steep slopes, ridgelines, exposed rock, snow-like light areas, and similar blue-gray-brown textures. At 150×150 resolution, fine distinctions between snow, ice, scree, and exposed rock may be weakened.

This explanation is plausible but not proven. The artifacts do not show whether resolution caused these errors.

Proposed validation experiment: Re-evaluate the same trained design at multiple input resolutions, such as 150×150, 224×224, and a higher resolution, using identical splits and repeated seeds. Test specifically whether glacier↔mountain errors decrease as fine surface detail is preserved.

Mixed scenes and contextual cues

Observed evidence: The error gallery shows mountainous scenes containing combinations of slopes, snow-like material, sky, and water. One glacier→mountain example receives 98.9% confidence, and one mountain→glacier example receives 98.5%.

Interpretation: The classifier may be relying on broad scene composition or texture rather than a precise distinction between glacial ice and mountain terrain. A glacier image dominated by rock may resemble the mountain class, while a snowy mountain may resemble the glacier class.

Proposed validation experiment: Create controlled crops or masks for the same images:

- Terrain-only
- Sky removed
- Water removed
- Snow/ice region masked
- Rock region masked

Compare probability changes. If masking snow/ice sharply reduces glacier probability, that would support—but still not conclusively prove—use of that cue.

Sea-related glacier errors

Observed evidence: Thirty glacier images are predicted as sea. Two glacier→sea examples appear in the Grad-CAM gallery at 99.6% and 98.6% confidence. Their heatmaps emphasize broad horizontal water/sky or shoreline-like regions.

Interpretation: Water, blue color, horizon structure, and reflective surfaces may dominate the prediction when the glacial component is small or visually weak. Some glacier images may also contain lakes or coastal-looking regions.

Proposed validation experiment: Compare errors on glacier images with and without visible water, using blinded human annotation. Estimate glacier recall separately for the two subgroups.

---

4. Street–buildings confusion

Urban co-occurrence

Observed evidence: Forty-nine of 501 street images are predicted as buildings, compared with 20 of 437 building images predicted as street. The supplied street→buildings example is wrong at 99.8% confidence.

Its Grad-CAM activation overlaps a prominent building/tower region rather than being restricted to the road surface.

Interpretation: Streets and buildings frequently coexist. If architecture occupies more of the frame or is more visually distinctive than the roadway, the model may favor buildings even when the folder label is street.

The directional asymmetry suggests that architectural evidence may dominate road evidence in some mixed urban scenes. This remains an interpretation, not a proven internal rule.

Proposed validation experiment: Measure façade area and visible-road area using manual masks or a segmentation model. Compare these ratios across:

- Correctly classified streets
- Street→buildings errors
- Correctly classified buildings
- Buildings→street errors

Test whether street→buildings errors contain systematically more façade area or less visible roadway.

Landmark and architectural dominance

Observed evidence: The 99.8%-confidence street→buildings example prominently contains a large clock tower and surrounding architecture. The Grad-CAM hotspot overlaps the architectural structure.

Interpretation: A distinctive landmark may be a stronger feature than the lower-level street region. The prediction may therefore be visually understandable even if it disagrees with the dataset label.

Proposed validation experiment: Mask the tower/building region and recompute predictions. Separately mask the road and pedestrian region. A larger probability change after architectural masking would support the landmark-dominance hypothesis.

---

5. Model error versus possible label ambiguity

Clear model error

Observed evidence: A prediction disagrees with the supplied folder label. For evaluation, every such disagreement is counted as an error.

Interpretation: It is defensible to call it a model error relative to the dataset ground truth. It is not always defensible to call it an unequivocal semantic error, because the images may contain multiple valid scene concepts.

Proposed validation experiment: Have several blinded reviewers assign:

- One dominant label
- All applicable labels
- An ambiguity score

A disagreement supported by strong reviewer consensus is more likely to be a genuine model error.

Possible glacier–mountain ambiguity

Observed evidence: The gallery includes glacier and mountain images containing overlapping terrain characteristics. The confusion is reciprocal rather than entirely one-directional.

Interpretation: Some errors may result from an under-specified boundary between glacier and mountain. For example, an image labeled glacier may be visually dominated by rock, while a mountain image may contain extensive snow or ice.

Reciprocal confusion is consistent with ambiguity, but it does not prove annotation ambiguity.

Proposed validation experiment: Ask domain-informed reviewers to relabel the 132 glacier↔mountain errors without seeing the original labels or model predictions. Report consensus and inter-rater agreement.

Possible street–buildings ambiguity

Observed evidence: The street→buildings example contains both a street-level scene and prominent architecture.

Interpretation: “Street” and “buildings” may describe different valid components of the same image. The dataset’s single-label structure forces one answer even when multiple concepts are present.

Proposed validation experiment: Compare single-label and multilabel annotation for urban error cases. If reviewers consistently select both street and buildings, part of the measured confusion reflects task formulation rather than purely defective recognition.

---

6. What Grad-CAM supports

Observed evidence: In the six selected confident errors, Grad-CAM highlights spatial regions associated with architecture, water/horizons, slopes, and snow- or rock-like terrain.

Interpretation: Grad-CAM supports the limited statement that the model's predicted-class score is locally sensitive to feature-map activations spatially associated with those regions. It helps generate hypotheses about possible cues.

Proposed validation experiment: Perform Grad-CAM sanity checks by progressively randomizing classifier and backbone weights. A meaningful explanation method should change substantially as learned parameters are destroyed.

---

7. What Grad-CAM cannot prove

Observed evidence: Only six deliberately selected, highly confident errors are visualized. No random errors, correct cases, alternative explanation methods, or perturbation tests are supplied.

Interpretation: The figure cannot prove:

- That highlighted pixels caused the prediction
- That non-highlighted regions were irrelevant
- That the model recognized concepts such as "ice," "road," or "building"
- That the explanations are faithful or stable
- That the same behavior occurs across the full test set
- That errors arise from bias rather than label ambiguity
- That the model is globally miscalibrated

The heatmaps are qualitative hypotheses, not causal explanations.

Proposed validation experiment: Compare Grad-CAM with occlusion sensitivity and an attribution method such as Integrated Gradients. Quantitatively test whether removing highly attributed regions reduces the predicted-class score more than removing equally sized random regions.

Three priority experiments

1. **Blinded ambiguity audit**
   Independently relabel glacier↔mountain and street↔buildings errors using multiple reviewers, multilabel choices, confidence ratings, and inter-rater agreement.
2. **Counterfactual region intervention**
   Mask or crop snow/ice, rock, water, road, and architecture regions. Measure changes in class probabilities and predictions against random-region controls.

3. **Controlled subgroup challenge set**
   Evaluate separately on:
   - Pure glacier versus glacier-with-rock/water
   - Pure mountain versus snowy mountain
   - Road-dominant streets versus architecture-dominant streets
   - Building-only versus building-with-visible-road scenes
4. This would test whether mixed-scene composition predicts the observed directional failures without inventing new performance claims.

PROMPT:

Design a deployment plan for the final MobileNetV2 classifier.

Include:

- Input validation
- Identical preprocessing
- Model versioning
- Probability calibration
- Confidence thresholds
- Handling images outside the six known classes
- Human-review routing
- Class-specific monitoring
- Input and concept drift
- Latency and resource considerations
- Feedback collection
- Retraining triggers
- Rollback criteria

Present the design as a concise architecture and monitoring table. Clearly label any recommendation that requires new experiments.

CHATGPT RESPONSE:

Deployment architecture

```
flowchart TD
    A["Image request"] --> B["Validate and decode"]
    B -->|Invalid| C["Reject with reason"]
    B -->|Valid| D["Resize to 150×150 RGB"]
    D --> E["Versioned MobileNetV2 model"]
    E --> F["Calibration layer"]
    F --> G{"OOD or low confidence?"}
    G -->|Yes| H["Human review / Unknown"]
    G -->|No| I["Return class and calibrated probability"]
```

```
H --> J["Verified feedback store"]
I --> K["Monitoring logs"]
J --> L["Evaluation and retraining pipeline"]
K --> L
```

The production artifact should contain or be paired immutably with the exact preprocessing, class order, calibration parameters, and decision-policy version.

Architecture table

Component	Deployment design	Verification or experiment
Input validation	Accept documented formats such as JPEG and PNG. Verify successful decoding, three-channel RGB conversion, finite pixel values, file-size and dimension limits, and protection against decompression bombs. Reject empty, corrupted, unsupported, or excessively large files with a clear error code.	Test malformed, grayscale, RGBA, CMYK, truncated, very small, and extremely large inputs before release.
Identical preprocessing	Convert to RGB, resize exactly to 150×150 using the same TensorFlow resize behavior, then apply `tf.keras.applications.mobilenet_v2.preprocess_input`. Do not apply random flip, rotation, zoom, or contrast during inference.	Store reference images and expected tensors/predictions; verify parity between notebook and serving pipeline.
Model artifact	Explicitly reload the chosen saved checkpoint and export a clean inference model. Confirm that it reproduces the reported predictions before deployment.	**New experiment required:** compare the saved best checkpoint with the final in-memory model because equivalence was not established in the existing artifacts.
Model versioning	Assign an immutable version containing model checksum, code version, TensorFlow version, class	Require prediction-regression tests before

	mapping, preprocessing version, calibration version, training-data fingerprint, and decision-policy version.	registering a version.
Output contract	Return model version, predicted class, calibrated probability, review/abstention status, and reason code. Keep the class order fixed: buildings, forest, glacier, mountain, sea, street.	Validate the mapping against known reference cases.
Probability calibration	Fit temperature scaling or another simple calibration method using validation data only. Never fit calibration parameters on the test set.	**New experiment required:** compare uncalibrated and calibrated NLL, Brier score, ECE, and reliability diagrams.
Confidence thresholds	Use calibrated probabilities. Define separate thresholds for automatic acceptance, human review, and abstention. Do not infer thresholds from the six selected confident errors.	**New experiment required:** select thresholds from validation coverage-risk curves and operational error costs.
Unknown classes	Treat “none of the six classes” as an abstention problem; softmax always chooses one known class. Combine low confidence with an out-of-distribution detector. Return `unknown/review_required`, not a forced label.	**New experiment required:** build an OOD validation set and compare maximum calibrated probability, entropy, energy scores, and embedding-distanc e methods.
Human-revie w routing	Route low-confidence, OOD, invalid-but-recoverable, and policy-sensitive cases to review. Give priority to glacier–mountain and street–buildings disagreements. Reviewers may select one class, multiple plausible classes, unknown, or labeling error.	Pilot the queue to measure review volume, agreement, and turnaround time.

Logging	Record request/model versions, predicted probabilities, decision status, latency, validation failures, drift features, and eventual reviewer labels. Avoid retaining raw images unless authorized and necessary.	Perform privacy, retention, and access-control review before launch.
Serving reliability	Use bounded request sizes, timeouts, concurrency limits, health checks, warm model loading, and graceful failure. Separate inference failures from abstentions.	Load-test the complete serving path.
Rollout	Begin with shadow evaluation, then a small canary deployment, followed by gradual promotion. Keep the previous model immediately deployable.	Promotion requires passing quality, calibration, latency, and error-budget gates.

Monitoring table

Monitoring area	Metrics	Action or trigger
Overall quality	Accuracy, macro-F1, weighted-F1, loss, reviewed error rate	Investigate statistically credible or operationally significant degradation relative to the approved baseline. Numerical limits require production baselining.
Class-specific quality	Per-class precision, recall, F1, support, and prediction frequency	Alert when a class deteriorates even if aggregate accuracy remains stable.
Critical confusions	Mountain→glacier, glacier→mountain, street→buildings, buildings→street, glacier→sea	Track directional counts and rates separately. Escalate sustained increases after controlling for changes in class prevalence.
Calibration	NLL, Brier score, ECE, reliability by probability bin and class	Recalibrate or retrain when confidence no longer corresponds to observed correctness. **New experiment required** to establish initial baselines.
Abstention behavior	Automatic-acceptance rate, human-review rate, OOD rate, unresolved rate	Investigate sudden increases, queue overload, or an unsafe decrease in abstention accompanied by more errors.

Input drift	Resolution, aspect ratio, brightness, contrast, blur, compression, color statistics, file format, embedding distribution	Compare against training and approved-production reference distributions. Investigate sustained drift before retraining.
Concept drift	Performance within stable input subgroups; class-conditional error and calibration changes	Suspect concept drift when image statistics remain similar but the relationship between inputs and verified labels changes.
Unknown content	Reviewer-confirmed unknown categories, OOD-score distribution, forced-label attempts	Expand rejection coverage or consider taxonomy changes when recurring unknown categories become operationally important.
Human review	Queue size, age, reviewer agreement, override rate, class-specific overrides	Audit high disagreement and repeated overrides; these may indicate label ambiguity or an obsolete taxonomy.
Latency	Median, p95 and p99 end-to-end latency; preprocessing and inference time separately	Roll back or scale when service objectives are breached persistently. **New experiment required** to set hardware-specific objectives.
Resources	CPU/GPU utilization, memory, batch size, throughput, cold-start time, error rate	Benchmark CPU and GPU serving; consider quantization only after measuring its effect on class-level accuracy and calibration.
Data quality	Decode failures, unsupported formats, extreme sizes, duplicate requests	Investigate upstream pipeline changes or abuse patterns.
Feedback quality	Percentage of predictions receiving verified labels, time to label, reviewer confidence	Do not retrain from unverified user clicks or implicit feedback without validation.

Retraining triggers

Retraining should be considered when one or more of these conditions persist over a sufficiently large verified sample:

- Class-specific recall or F1 declines beyond an approved tolerance.
- Glacier–mountain or street–buildings confusion increases materially.
- Calibration error rises enough to invalidate confidence thresholds.

- OOD or human-review volume exceeds operational capacity.
- A recurring unknown category warrants taxonomy expansion.
- Input or concept drift is confirmed.
- Sufficient high-quality reviewed examples accumulate to support an update.

Exact numerical triggers require new experiments and production baseline data. Retraining should not be triggered by a small number of unverified complaints.

Rollback criteria

Roll back to the last approved version if a new release causes:

- A verified regression in overall or class-specific quality
- Worse glacier–mountain or street–buildings confusion
- Calibration failure that makes automatic acceptance unsafe
- Unexpected class-order or preprocessing mismatch
- Prediction differences from reference regression tests
- Excessive latency, memory use, crashes, or timeouts
- A large unexplained change in abstention or class distribution
- Failure of logging, auditability, or human-review routing

Rollback should restore the model, preprocessing, calibration, and threshold policy together; these components form one deployable version.

PROMPT:

Using only the verified and corrected analysis from this conversation, create:

1. A concise GitHub project summary
2. A publication-ready Medium article outline
3. A reproducibility statement
4. An AI-assistance disclosure
5. A section explaining how iterative prompt engineering improved the project

Explicitly describe the prompting techniques demonstrated in this session, including role prompting, context grounding, constraint specification, staged decomposition, output contracts, evidence requirements, adversarial self-critique, and iterative refinement.

Do not introduce metrics that were not verified earlier.

CHATGPT RESPONSE:

1. Concise GitHub project summary

This project compares a custom convolutional neural network with a fine-tuned MobileNetV2 for classifying Intel scene images into six categories: buildings, forest, glacier, mountain, sea, and street.

The custom CNN achieved 80.10% test accuracy with a loss of 0.5698. Fine-tuned MobileNetV2 achieved 90.27% test accuracy with a loss of 0.2487 and a macro-F1 score of 0.9048. Model-failure analysis found that glacier–mountain and street–buildings were the most important confusion patterns. Grad-CAM was used to examine spatial sensitivity in selected high-confidence errors, while recognizing that these visualizations do not establish causal or human-like reasoning.

The repository includes the executed TensorFlow notebook, evaluation tables, confusion matrices, Grad-CAM visualizations, and machine-readable result summaries. It also documents reproducibility limitations, artifact inconsistencies, calibration requirements, and a deployment-monitoring design.

2. Publication-ready Medium article outline

Proposed title

Teaching a Neural Network to Recognize Natural Scenes: From a Custom CNN to Explainable Transfer Learning

Subtitle

A reproducible comparison of a custom TensorFlow CNN and fine-tuned MobileNetV2 on the Intel Image Classification dataset—with failure analysis, Grad-CAM, and lessons from iterative prompt engineering.

1. Introduction: Why scene classification is harder than it looks

- Introduce the six classes: buildings, forest, glacier, mountain, sea, and street.
- Explain that several labels can coexist visually within one image.
- State the project objective: compare a custom CNN with an ImageNet-initialized MobileNetV2.
- Clarify that the analysis focuses not only on accuracy but also on where and how the final model fails.

2. Framing the project with CRISP-DM

Organize the workflow around:

1. Business Understanding
2. Data Understanding

3. Data Preparation
4. Modeling
5. Evaluation
6. Deployment

Explain that the original notebook primarily addresses the technical classification problem; a real deployment would additionally require operational objectives, error costs, calibration, and monitoring.

3. Understanding the dataset

- 14,034 labeled images in the original training directory
- 11,228 images used for training
- 2,806 images used for validation
- 3,000 images used for testing
- Six classes with limited but nonzero class imbalance
- One representative image per class and a class-distribution visualization

Discuss limitations:

- No exact or perceptual duplicate audit
- No file-level split manifest
- No systematic label-ambiguity review
- No external-distribution evaluation

4. Preparing the images

- Resize images to 150×150
- Use a batch size of 64
- Apply horizontal flipping, rotation, zoom, and contrast augmentation during training
- Rescale custom-CNN inputs to [0,1][0,1]
- Use `mobilenet_v2.preprocess_input` for MobileNetV2
- Keep the test dataset ordered with `shuffle=False`

Clarify that identical serving-time preprocessing is essential for deployment.

5. Model 1: Building a custom CNN

Describe:

- Four convolutional blocks
- Batch Normalization and max pooling
- Global average pooling
- A 128-unit dense layer
- 40% dropout

- 424,006 total parameters
- Adam optimization, early stopping, learning-rate reduction, and checkpointing

Report the verified test result:

- Accuracy: 80.10%
- Loss: 0.5698

Avoid diagnosing overfitting solely from non-epoch-aligned maximum training and validation accuracies.

6. Model 2: Transfer learning with MobileNetV2

Describe:

- ImageNet initialization
- Initially frozen convolutional base
- Global average pooling and 35% dropout
- Partial fine-tuning of the final portion of MobileNetV2
- Frozen Batch Normalization layers
- Fine-tuning learning rate of 10−510^{-5}

Explain that this staged design is intended to preserve pretrained representations while adapting higher-level features. State clearly that its benefit was not isolated through ablation.

7. Verified model comparison

Model	Test accuracy	Test loss	Macro-F1
Custom CNN	80.10%	0.5698	Not computed
Fine-tuned MobileNetV2	90.27%	0.2487	0.9048

Explain that MobileNetV2 achieved higher accuracy and lower loss in this run.

Do not claim that model complexity alone caused the improvement. Architecture, ImageNet initialization, preprocessing, optimization, parameter count, and training schedule all changed simultaneously.

8. Looking beyond overall accuracy

Present the MobileNetV2 class ranking by F1:

1. Forest: 0.9895
2. Sea: 0.9323

3. Street: 0.9169
4. Buildings: 0.9161
5. Mountain: 0.8424
6. Glacier: 0.8315

Clarify that street only narrowly exceeds buildings and that no uncertainty estimates are available.

9. Where the model fails

Present the largest directional confusions:

- Mountain → glacier: 69 images, 13.1%
- Glacier → mountain: 63 images, 11.4%
- Street → buildings: 49 images, 9.8%
- Glacier → sea: 30 images, 5.4%
- Buildings → street: 20 images, 4.6%

Discuss plausible visual explanations:

- Snow, ice, rock, ridgelines, and terrain texture overlap between glacier and mountain.
- Roads and buildings naturally coexist in urban images.
- Water and horizon cues may influence glacier → sea errors.

Label these explanations as hypotheses rather than established causes.

10. Model error or label ambiguity?

Explain the difference between:

- An evaluation error: prediction differs from the folder label
- An unequivocal semantic error: human reviewers agree that the prediction is visually wrong

Use mixed urban and mountain/glacier scenes to show why single-label classification can oversimplify the image.

Recommend blinded multi-reviewer relabeling and multilabel annotation as future validation.

11. What the confident errors reveal

Discuss the six displayed errors with confidence between 98.5% and 99.8%.

Correct interpretation:

- Some wrong predictions receive very high softmax scores.

- High softmax probability should not automatically be interpreted as a trustworthy correctness probability.

Important limitation:

- The examples were intentionally selected as confident mistakes.
- They do not prove that the entire model is globally miscalibrated.

12. What Grad-CAM shows—and what it does not

Grad-CAM supports a limited claim: the predicted-class score is sensitive to feature-map activations associated with particular image regions.

It cannot prove:

- Causal feature use
- Human-like concept recognition
- Explanation faithfulness
- Explanation stability
- Global model behavior
- Whether the dataset label is semantically unambiguous

Recommend occlusion tests, attribution-method comparisons, and explanation sanity checks.

13. An artifact-consistency lesson

Explain that the confusion-matrix image and exported report support:

- Mountain → glacier: 69
- Glacier → mountain: 63
- Street → buildings: 49

The source of one notebook cell instead contains 72, 52, and 50. The cause cannot be reconstructed from the artifacts.

Use this as a reproducibility lesson: derived summaries should be calculated directly from saved predictions rather than manually hard-coded.

14. From notebook to deployment

Describe the proposed production flow:

- Validate and decode inputs
- Convert to RGB
- Resize to 150×150
- Apply identical MobileNetV2 preprocessing

- Produce probabilities
- Apply a validation-fitted calibration layer
- Accept, abstain, or route to human review
- Monitor class-specific performance and drift

Mark calibration thresholds, OOD detection, latency requirements, and monitoring limits as requiring new experiments.

15. How iterative prompt engineering improved the project

Explain the progression:

- Start with a domain-role prompt
- Ground the analysis in attached artifacts
- Impose nonfabrication constraints
- Decompose the audit through CRISP-DM
- Require explicit output structures
- Challenge the first audit through skeptical peer review
- Narrow subsequent prompts to failure analysis and deployment
- Refine claims from broad interpretations to evidence-calibrated conclusions

Emphasize that prompt engineering improved analytical discipline and documentation, not the already-measured model scores.

16. Conclusion

- MobileNetV2 achieved stronger verified test performance in this run.
- Glacier–mountain confusion remains the central failure mode.
- Confident errors motivate calibration and abstention.
- Grad-CAM is useful for hypothesis generation but insufficient as causal evidence.
- Reproducibility requires saved predictions, exact split manifests, checkpoint verification, environment capture, and repeated runs.

---

3. Reproducibility statement

Reproducibility Statement

The executed experiment used TensorFlow 2.20.0, two Tesla T4 GPUs, a random seed of 42, an image size of 150×150, and a batch size of 64. Python, NumPy, and TensorFlow random seeds were set in the notebook. The labeled dataset was divided into 11,228 training images and 2,806 validation images, while a separate ordered test directory containing 3,000 images was used for final evaluation.

The notebook records the model architectures, preprocessing functions, augmentation layers, optimizers, learning rates, callbacks, training outputs, and evaluation procedure. The exported artifacts include model-comparison results, a MobileNetV2 classification report, confusion matrices, Grad-CAM visualizations, and a JSON summary.

These details improve repeatability but do not guarantee exact reproduction. The artifacts do not include file-level split manifests, dataset checksums, a complete dependency lock file, CUDA or cuDNN versions, deterministic-operation settings, saved-model checksums, or results from repeated seeds. Multi-GPU execution, GPU kernels, data prefetching, and stochastic augmentation may introduce nondeterminism.

One notebook source cell also contains manually specified confusion counts that disagree with the exported confusion matrix and JSON summary. The counts supported by the matrix, normalized percentages, class supports, recall values, and overall accuracy were used in the final analysis. The cause of the discrepancy cannot be determined from the available artifacts.

A fully reproducible release should preserve exact split manifests, sample-level predictions, environment metadata, dataset and model checksums, deterministic settings, and repeated-run results. The selected MobileNetV2 checkpoint should also be explicitly reloaded and verified against stored reference predictions before publication or deployment.

4. AI-assistance disclosure

AI-Assistance Disclosure

Generative AI was used as a technical-assistance and review tool during this project. It supported the structured audit of the executed notebook and exported results, identification of artifact inconsistencies, interpretation of model-failure patterns, development of reproducibility and deployment recommendations, and refinement of the project documentation.

The AI was instructed to treat the executed notebook, CSV files, JSON summary, confusion matrix, and Grad-CAM figure as the only authoritative evidence. It was explicitly prohibited from inventing or estimating experimental results. Numerical claims included in the final documentation were checked against the supplied artifacts, and disagreements between artifacts were disclosed rather than silently resolved.

AI-generated interpretations—such as possible visual explanations for glacier–mountain or street–buildings confusion—are presented as hypotheses, not experimentally established causes. Proposed calibration, out-of-distribution detection, robustness testing, repeated-seed evaluation, and deployment thresholds require new experiments and are not reported as completed results.

The author remains responsible for verifying the analysis, interpreting the findings, maintaining academic integrity, and ensuring that all published claims accurately reflect the executed experiment.

5. How iterative prompt engineering improved the project

How Iterative Prompt Engineering Improved the Project

Prompt engineering was used as an iterative analytical process rather than as a single request for polished writing. Each prompt introduced a clearer role, stronger evidence boundary, or more precise output requirement. This progressively changed the AI's contribution from a general notebook summary into a skeptical, traceable review of model performance, failure modes, reproducibility, and deployment readiness.

Role prompting

The initial prompt assigned the AI several relevant roles: senior computer-vision scientist, TensorFlow engineer, peer reviewer, technical educator, and CRISP-DM specialist. This established the expected technical depth and encouraged the analysis to address both implementation and scientific validity.

Role prompting did not make the AI's conclusions automatically correct. Its value was to define the perspective from which the evidence should be examined.

Context grounding

The AI was grounded in the executed notebook, CSV files, JSON summary, confusion matrix, and Grad-CAM figure. These artifacts were declared the only authoritative evidence.

This prevented the analysis from substituting expected benchmark performance, remembered dataset statistics, or unexecuted assumptions for the project's actual results. Context grounding also made it possible to detect that one notebook source cell contained confusion counts that disagreed with the exported artifacts.

Constraint specification

The prompts imposed explicit constraints:

- Do not fabricate or estimate results.
- Identify disagreements between artifacts.
- Separate measured evidence, inference, and recommendation.
- Do not rewrite the Medium article before completing the audit.
- Do not introduce unverified metrics.
- Mark deployment recommendations that require new experiments.

These constraints reduced unsupported certainty and made the boundary between completed work and future work visible.

Staged decomposition

The work was divided into successive stages instead of requesting everything at once:

1. Conduct a CRISP-DM audit.
2. Critique that audit as a skeptical peer reviewer.
3. Correct unsupported or overly strong claims.
4. Focus specifically on model failures.
5. Design deployment and monitoring controls.
6. Convert the verified analysis into publication materials.

This staged decomposition allowed each response to become evidence for the next review step. It also made methodological weaknesses easier to detect than they would have been in a single broad writing prompt.

Output contracts

Each prompt specified the required structure of the response. Examples included:

- Six CRISP-DM stages
- A verified results table
- Lists of findings, limitations, and improvements
- A claim-audit table with fixed columns
- Explicit “Observed evidence,” “Interpretation,” and “Proposed validation experiment” labels
- A concise architecture and monitoring table

These output contracts made omissions visible and forced conclusions to be paired with their evidentiary basis.

Evidence requirements

The AI was required to cite exact notebook cells, metrics, tables, and images. It was also required to resolve artifact disagreements explicitly.

This requirement exposed an important reproducibility issue: the confusion counts written in one notebook source cell did not match the confusion matrix and exported JSON. The final analysis therefore used the values that were internally consistent with the matrix, normalized percentages, class supports, recalls, and overall accuracy while preserving the discrepancy as an unresolved provenance problem.

Adversarial self-critique

After the first audit, the AI was asked to review its own work as a skeptical peer reviewer. It classified earlier statements as measured, inferred, or unsupported and identified claims that needed correction.

For example, the initial language around overfitting, class separability, checkpoint identity, and calibration was softened because the existing artifacts did not fully establish those conclusions. This adversarial step reduced the risk that plausible interpretations would be mistaken for experimental findings.

Iterative refinement

Later prompts narrowed the analysis to specific questions:

- Which classes were strongest and weakest?
- What were the largest directional confusions?
- Which visual explanations were plausible?
- Which cases might involve label ambiguity?
- What could Grad-CAM support?
- What experiments would test those explanations?
- What safeguards would be required for deployment?

This refinement converted broad observations into testable hypotheses. Glacier–mountain similarity, architectural dominance in street images, probability calibration, and out-of-distribution handling were presented as questions for further experimentation rather than settled facts.

Overall, iterative prompt engineering improved the rigor, traceability, and communication of the project. It did not change the already-executed model or generate new experimental results. Its contribution was to make the analysis more disciplined: claims were tied to evidence, uncertainty was made explicit, inconsistencies were preserved, and future recommendations were separated from verified findings.